```python
import numpy as np
import matplotlib.pyplot as plt
from dataclasses import dataclass
from matplotlib.animation import FuncAnimation
from matplotlib.widgets import Slider, Button, RadioButtons, CheckButtons


# =========================
#  Model definitions
# =========================

@dataclass
class BridgeParams:
    # Bridge
    M: float = 1.0
    f0_hz: float = 1.0
    zeta: float = 0.02

    # Crowd coupling
    G: float = 0.02
    C: float = 1.5
    alpha: float = np.pi / 2

    # Walker frequencies (rad/s)
    detune: float = 0.0      # mean shift as fraction of Omega
    sigma: float = 0.35      # std dev (rad/s)

    @property
    def Omega(self) -> float:
        return 2.0 * np.pi * self.f0_hz

    @property
    def K(self) -> float:
        return self.M * self.Omega ** 2

    @property
    def B(self) -> float:
        return 2.0 * self.zeta * self.Omega * self.M

    @property
    def omega_mean(self) -> float:
        return self.Omega * (1.0 + self.detune)


def amp_phase(X: float, V: float, Omega: float):
    """Return amplitude A and phase Psi from displacement X and velocity V."""
    if Omega <= 0:
        return 0.0, 0.0
    Vn = V / Omega
    A = float(np.hypot(X, Vn))
    Psi = float(np.arctan2(X, Vn))  # X = A sin Psi, V/Omega = A cos Psi
    return A, Psi


def order_param(theta: np.ndarray):
    """Kuramoto order parameter magnitude R and angle Phi."""
    z = np.mean(np.exp(1j * theta))
    return float(np.abs(z)), float(np.angle(z))


class MillenniumBridgeSim:
    """
    Real-time friendly integrator (vectorized RK4).
    State: (X, V, theta[0..N-1]) with fixed omega.
    """
    def __init__(self, N: int, params: BridgeParams, seed: int = 42):
        self.rng = np.random.default_rng(seed)
        self.params = params
        self.reset(N)

    def reset(self, N: int):
        self.N = int(N)
        p = self.params
        self.t = 0.0
        self.X = 1e-6 * self.rng.normal()
        self.V = 1e-6 * self.rng.normal()
        self.theta = self.rng.uniform(-np.pi, np.pi, size=self.N)
        self.omega = self.rng.normal(
            loc=p.omega_mean,
            scale=max(p.sigma, 1e-12),
            size=self.N
        )

    def _deriv(self, X: float, V: float, theta: np.ndarray):
        p = self.params
        Om = p.Omega
```

```
        A, Psi = amp_phase(X, V, Om)
        F = p.G * float(np.sin(theta).sum())
        dX = V
        dV = (-p.B * V - p.K * X + F) / p.M
        dtheta = self.omega + p.C * A * np.sin(Psi - theta + p.alpha)
        return dX, dV, dtheta

    def rk4_step(self, dt: float):
        X0, V0, th0 = self.X, self.V, self.theta

        k1x, k1v, k1t = self._deriv(X0, V0, th0)

        X1 = X0 + 0.5 * dt * k1x
        V1 = V0 + 0.5 * dt * k1v
        th1 = th0 + 0.5 * dt * k1t
        k2x, k2v, k2t = self._deriv(X1, V1, th1)

        X2 = X0 + 0.5 * dt * k2x
        V2 = V0 + 0.5 * dt * k2v
        th2 = th0 + 0.5 * dt * k2t
        k3x, k3v, k3t = self._deriv(X2, V2, th2)

        X3 = X0 + dt * k3x
        V3 = V0 + dt * k3v
        th3 = th0 + dt * k3t
        k4x, k4v, k4t = self._deriv(X3, V3, th3)

        self.X = X0 + (dt / 6.0) * (k1x + 2*k2x + 2*k3x + k4x)
        self.V = V0 + (dt / 6.0) * (k1v + 2*k2v + 2*k3v + k4v)
        self.theta = th0 + (dt / 6.0) * (k1t + 2*k2t + 2*k3t + k4t)
        self.t += dt

    def step(self, dt: float, dt_int: float = 0.01):
        if dt <= 0:
            return
        n = max(1, int(np.ceil(dt / max(dt_int, 1e-6))))
        h = dt / n
        for _ in range(n):
            self.rk4_step(h)


# =========================
#  Theory helpers
# =========================

def normal_pdf(x: float, mu: float, sigma: float) -> float:
    sigma = max(sigma, 1e-12)
    return (1.0 / (sigma * np.sqrt(2.0 * np.pi))) * np.exp(-0.5 * ((x - mu) / sigma) ** 2)


def critical_crowd_size(params: BridgeParams) -> float:
    # Nc = 2 B Omega / (pi G C P(Omega))
    P = normal_pdf(params.Omega, params.omega_mean, max(params.sigma, 1e-12))
    denom = np.pi * max(params.G, 1e-12) * max(params.C, 1e-12) * max(P, 1e-18)
    return 2.0 * params.B * params.Omega / denom


# =========================
#  Adaptive bifurcation recorder
# =========================

class BifurcationRecorder:
    """
    Stores points as full snapshots so changing axes is instantaneous and consistent.
    When a (tracked) parameter changes, we:
      1) wait settle_s
      2) wait transient_s
      3) for sample_s, detect peaks of chosen y-metric and store multiple points
      If steady (no peaks), store mean of tail.
    """
    X_KEYS = ("C", "G", "ζ", "N", "M", "f0")
    Y_KEYS = ("R", "A")

    def __init__(self):
        self.points = []  # list[dict]
        self.x_key = "C"
        self.y_key = "R"

        self.enabled = True
        self.settle_s = 0.25
        self.transient_s = 1.25
        self.sample_s = 2.75

        self._state = "idle"
        self._t_changed = 0.0
        self._t_state = 0.0
        self._x_val = np.nan
```

```
        self._y_series = []
        self._points_at_sample_start = 0

    def clear(self):
        self.points.clear()
        self._state = "idle"
        self._y_series.clear()
        self._points_at_sample_start = 0

    def set_axes(self, x_key: str = None, y_key: str = None):
        if x_key is not None:
            self.x_key = x_key
        if y_key is not None:
            self.y_key = y_key

    def mark_changed(self, t_now: float, x_val: float):
        if not self.enabled:
            return
        self._state = "settle"
        self._t_changed = float(t_now)
        self._t_state = float(t_now)
        self._x_val = float(x_val)
        self._y_series.clear()

    def manual_add(self, snap: dict):
        self.points.append(dict(snap))

    def tick_and_maybe_record(self, t_now: float, snap: dict, y_val: float):
        if not self.enabled:
            return

        t_now = float(t_now)

        if self._state == "settle":
            if (t_now - self._t_changed) >= self.settle_s:
                self._state = "transient"
                self._t_state = t_now
                self._y_series.clear()

        elif self._state == "transient":
            if (t_now - self._t_state) >= self.transient_s:
                self._state = "sample"
                self._t_state = t_now
                self._y_series.clear()
                self._points_at_sample_start = len(self.points)

        elif self._state == "sample":
            self._y_series.append(float(y_val))

            # peak detection: y[i-1] > y[i-2] and y[i-1] > y[i]
            if len(self._y_series) >= 3:
                y0, y1, y2 = self._y_series[-3], self._y_series[-2], self._y_series[-1]
                if (y1 > y0) and (y1 > y2):
                    p = dict(snap)
                    p["bif_x"] = self._x_val
                    p["bif_y"] = float(y1)
                    self.points.append(p)

            if (t_now - self._t_state) >= self.sample_s:
                if len(self.points) == self._points_at_sample_start and len(self._y_series):
                    tail = self._y_series[-min(60, len(self._y_series)):]
                    p = dict(snap)
                    p["bif_x"] = self._x_val
                    p["bif_y"] = float(np.mean(tail))
                    self.points.append(p)

                self._state = "idle"
                self._y_series.clear()

    def projected_xy(self):
        xs, ys = [], []
        for p in self.points:
            if ("bif_x" in p) and ("bif_y" in p):
                x = p["bif_x"]
                y = p["bif_y"]
            else:
                x = p.get(self.x_key, np.nan)
                y = p.get(self.y_key, np.nan)
            if np.isfinite(x) and np.isfinite(y):
                xs.append(float(x))
                ys.append(float(y))
        if not xs:
            return np.empty((0,)), np.empty((0,))
        return np.array(xs), np.array(ys)


# ========================
```

```python
#  Interactive UI
# ========================

def run_interactive():
    params = BridgeParams()
    sim = MillenniumBridgeSim(N=200, params=params, seed=42)

    fps = {"value": 22.0}
    sim_speed = {"value": 1.0}   # simulated seconds per real second
    dt_int = {"value": 0.01}     # integration substep

    paused = {"value": False}
    show_phases = {"value": True}

    # rolling history
    max_pts = 3000
    t_hist = np.full(max_pts, np.nan)
    x_hist = np.full(max_pts, np.nan)
    a_hist = np.full(max_pts, np.nan)
    r_hist = np.full(max_pts, np.nan)
    head = 0
    history_window_s = {"value": 18.0}

    # adaptive bifurcation
    bif = BifurcationRecorder()
    bif_last_count = {"n": 0}  # reduce redraw churn

    # QUEUE/LOCK for bif-x parameter changes while recorder is busy:
    # If Auto bif is enabled and the recorder is not idle, we queue the latest requested
    # value for the *current* bif.x_key and apply it only when the recorder finishes.
    x_queue = {"key": None, "val": None, "needs_reset": False}

    # style
    plt.rcParams.update({
        "axes.titlesize": 11,
        "axes.labelsize": 10,
        "xtick.labelsize": 9,
        "ytick.labelsize": 9,
        "figure.titlesize": 13,
    })

    # Bigger canvas + more bottom space for controls (Mac-friendly)
    fig = plt.figure(figsize=(16.5, 9.5))
    try:
        mgr = plt.get_current_fig_manager()
        if hasattr(mgr, "resize"):
            mgr.resize(1650, 950)
    except Exception:
        pass

    # Plots (top 2/3). Controls live in bottom ~1/3.
    ax_circle = fig.add_axes([0.05, 0.62, 0.36, 0.33])
    ax_xtime  = fig.add_axes([0.46, 0.62, 0.50, 0.33])
    ax_bif    = fig.add_axes([0.05, 0.37, 0.36, 0.20])
    ax_rtime  = fig.add_axes([0.46, 0.37, 0.50, 0.20])

    title = fig.suptitle("Millennium Bridge: damped mode + phase oscillators", y=0.97)

    # Phase circle
    ax_circle.set_title("Pedestrian phases")
    ax_circle.set_xlim(-1.2, 1.2)
    ax_circle.set_ylim(-1.2, 1.2)
    ax_circle.set_aspect("equal", adjustable="box")
    ax_circle.set_xticks([])
    ax_circle.set_yticks([])
    ax_circle.add_patch(plt.Circle((0, 0), 1.0, fill=False, lw=1.5, color="lightgray"))
    phase_sc = ax_circle.scatter([], [], s=8, alpha=0.55)
    (op_line,) = ax_circle.plot([], [], lw=2.5)
    (op_pt,) = ax_circle.plot([], [], "o", markersize=5)

    # Bridge displacement time series + envelope
    ax_xtime.set_title("Bridge motion")
    ax_xtime.set_xlabel("time (s)")
    ax_xtime.set_ylabel("X(t)")
    ax_xtime.grid(True, alpha=0.25)
    (line_x,) = ax_xtime.plot([], [], lw=1.7, label="X(t)")
    (line_ap,) = ax_xtime.plot([], [], lw=1.1, linestyle=":", label="+A(t)")
    (line_am,) = ax_xtime.plot([], [], lw=1.1, linestyle=":", label="-A(t)")
    ax_xtime.legend(loc="upper right", framealpha=0.9, fontsize=9)

    # R time series
    ax_rtime.set_title("Synchrony")
    ax_rtime.set_xlabel("time (s)")
    ax_rtime.set_ylabel("R(t)")
    ax_rtime.set_ylim(0, 1.05)
    ax_rtime.grid(True, alpha=0.25)
    (line_r,) = ax_rtime.plot([], [], lw=2.0)
```

```
# Bifurcation scatter
ax_bif.set_title("Adaptive bifurcation sketch")
ax_bif.set_xlabel(bif.x_key)
ax_bif.set_ylabel(bif.y_key)
ax_bif.grid(True, alpha=0.25)
bif_sc = ax_bif.scatter([], [], s=18, alpha=0.85, c="red")
theory_line = ax_bif.axvline(np.nan, color="black", linestyle="--", lw=2.0, alpha=0.85, label="Theory (critical)")
ax_bif.legend(loc="upper left", framealpha=0.9, fontsize=9)

# phase scatter downsampling
display_idx = {"idx": np.arange(min(sim.N, 450))}
def refresh_display_idx():
    if sim.N <= 450:
        display_idx["idx"] = np.arange(sim.N)
    else:
        display_idx["idx"] = np.linspace(0, sim.N - 1, 450).astype(int)
refresh_display_idx()

# --------- helpers for bif axes limits (avoid expensive autoscale churn) ----------
def set_bif_x_limits_for_key(key: str):
    # Match slider ranges (keeps bif plot stable and fast).
    if key == "C":
        ax_bif.set_xlim(0.0, 3.0)
    elif key == "G":
        ax_bif.set_xlim(0.0, 0.10)
    elif key == "ζ":
        ax_bif.set_xlim(0.0, 0.20)
    elif key == "N":
        ax_bif.set_xlim(10.0, 800.0)
    elif key == "M":
        ax_bif.set_xlim(0.2, 5.0)
    elif key == "f0":
        ax_bif.set_xlim(0.2, 2.5)
    else:
        ax_bif.set_xlim(0.0, 1.0)

def critical_value_for_xkey(x_key: str) -> float:
    # Returns the theoretical critical x-value (where Nc == N), for the current x-axis choice.
    # Uses Nc = 2 B Omega / (pi G C P(Omega)) and algebraic rearrangements where possible.
    Om = params.Omega
    P = normal_pdf(Om, params.omega_mean, max(params.sigma, 1e-12))
    Nnow = float(sim.N)

    # avoid division by zero nonsense
    G = max(float(params.G), 1e-12)
    C = max(float(params.C), 1e-12)
    zeta = max(float(params.zeta), 1e-12)
    M = max(float(params.M), 1e-12)
    P = max(float(P), 1e-18)
    Om2 = max(float(Om * Om), 1e-18)

    if x_key == "N":
        return float(critical_crowd_size(params))

    if x_key == "C":
        # Ccrit = 4 zeta Omega^2 M / (pi G P N)
        return float(4.0 * zeta * Om2 * M / (np.pi * G * P * Nnow))

    if x_key == "G":
        # Gcrit = 4 zeta Omega^2 M / (pi C P N)
        return float(4.0 * zeta * Om2 * M / (np.pi * C * P * Nnow))

    if x_key == "ζ":
        # zetacrit = N pi G C P / (4 Omega^2 M)
        return float(Nnow * np.pi * G * C * P / (4.0 * Om2 * M))

    if x_key == "M":
        # Mcrit = N pi G C P / (4 zeta Omega^2)
        return float(Nnow * np.pi * G * C * P / (4.0 * zeta * Om2))

    if x_key == "f0":
        # Solve for f0crit numerically in slider range [0.2, 2.5]
        f_lo, f_hi = 0.2, 2.5

        def Nc_minus_N(f0):
            Om_ = 2.0 * np.pi * f0
            B_ = 2.0 * params.zeta * Om_ * params.M
            omega_mean_ = Om_ * (1.0 + params.detune)
            P_ = normal_pdf(Om_, omega_mean_, max(params.sigma, 1e-12))
            denom = np.pi * max(params.G, 1e-12) * max(params.C, 1e-12) * max(P_, 1e-18)
            Nc_ = 2.0 * B_ * Om_ / denom
            return Nc_ - Nnow

        xs = np.linspace(f_lo, f_hi, 200)
        fs = np.array([Nc_minus_N(x) for x in xs])
        sgn = np.sign(fs)
```

```
            # find a sign change bracket
            idx = np.where(sgn[:-1] * sgn[1:] < 0)[0]
            if len(idx) == 0:
                return np.nan

            a, b = xs[idx[0]], xs[idx[0] + 1]
            fa, fb = Nc_minus_N(a), Nc_minus_N(b)

            # bisection
            for _ in range(40):
                m = 0.5 * (a + b)
                fm = Nc_minus_N(m)
                if fa * fm <= 0:
                    b, fb = m, fm
                else:
                    a, fa = m, fm
            return 0.5 * (a + b)

        return np.nan

    def set_bif_y_limits_for_key(key: str):
        if key == "R":
            ax_bif.set_ylim(0.0, 1.05)
        else:
            # A(t) can vary a lot; keep a reasonable default, adjust gently if needed.
            ax_bif.set_ylim(0.0, 0.25)

    set_bif_x_limits_for_key(bif.x_key)
    set_bif_y_limits_for_key(bif.y_key)


# Left-side buttons (bottom-left, not on top of sliders)
    ax_pause    = fig.add_axes([0.01, 0.27, 0.08, 0.05])
    ax_reset    = fig.add_axes([0.01, 0.21, 0.08, 0.05])
    ax_clear    = fig.add_axes([0.01, 0.15, 0.08, 0.05])
    ax_snap     = fig.add_axes([0.01, 0.09, 0.08, 0.05])
    ax_controls = fig.add_axes([0.01, 0.03, 0.08, 0.04])

    b_pause    = Button(ax_pause, "Pause")
    b_reset    = Button(ax_reset, "Reset")
    b_clear    = Button(ax_clear, "Clear\nbif")
    b_snap     = Button(ax_snap,  "Record")
    b_controls = Button(ax_controls, "Controls")

    # Main sliders (left column)
    slider_left = 0.12
    slider_w    = 0.50
    slider_h    = 0.018
    y0          = 0.30
    dy          = 0.028

    axN   = fig.add_axes([slider_left, y0 - 0*dy, slider_w, slider_h])
    axM   = fig.add_axes([slider_left, y0 - 1*dy, slider_w, slider_h])
    axz   = fig.add_axes([slider_left, y0 - 2*dy, slider_w, slider_h])
    axf   = fig.add_axes([slider_left, y0 - 3*dy, slider_w, slider_h])
    axC   = fig.add_axes([slider_left, y0 - 4*dy, slider_w, slider_h])
    axG   = fig.add_axes([slider_left, y0 - 5*dy, slider_w, slider_h])
    axa   = fig.add_axes([slider_left, y0 - 6*dy, slider_w, slider_h])
    axs   = fig.add_axes([slider_left, y0 - 7*dy, slider_w, slider_h])
    axd   = fig.add_axes([slider_left, y0 - 8*dy, slider_w, slider_h])
    axwin = fig.add_axes([slider_left, y0 - 9*dy, slider_w, slider_h])

    sN   = Slider(axN,   "N", 10, 800, valinit=sim.N, valstep=1)
    sM   = Slider(axM,   "mass M", 0.2, 5.0, valinit=params.M, valstep=0.05)
    sz   = Slider(axz,   "damping ζ", 0.0, 0.20, valinit=params.zeta, valstep=0.001)
    sf   = Slider(axf,   "bridge f0 (Hz)", 0.2, 2.5, valinit=params.f0_hz, valstep=0.01)
    sC   = Slider(axC,   "coupling C", 0.0, 6.0, valinit=params.C, valstep=0.01)
    sG   = Slider(axG,   "force G", 0.0, 0.10, valinit=params.G, valstep=0.001)
    sa   = Slider(axa,   "lag α", 0.0, np.pi, valinit=params.alpha, valstep=0.01)
    ssig = Slider(axs,   "cadence σ (rad/s)", 0.0, 2.0, valinit=params.sigma, valstep=0.01)
    sdet = Slider(axd,   "detune (fraction of Ω)", -0.30, 0.30, valinit=params.detune, valstep=0.005)
    swin = Slider(axwin, "history window (s)", 5.0, 60.0, valinit=history_window_s["value"], valstep=1.0)

    # Perf + bif tuning sliders (middle column)
    mid_left = 0.64
    mid_w    = 0.18
    mid_h    = 0.018
    mid_y0   = 0.30
    mid_dy   = 0.032

    axfps  = fig.add_axes([mid_left, mid_y0 - 0*mid_dy, mid_w, mid_h])
    axspd  = fig.add_axes([mid_left, mid_y0 - 1*mid_dy, mid_w, mid_h])
    axdti  = fig.add_axes([mid_left, mid_y0 - 2*mid_dy, mid_w, mid_h])
    axsett = fig.add_axes([mid_left, mid_y0 - 3*mid_dy, mid_w, mid_h])
    axtran = fig.add_axes([mid_left, mid_y0 - 4*mid_dy, mid_w, mid_h])
    axsam  = fig.add_axes([mid_left, mid_y0 - 5*mid_dy, mid_w, mid_h])
```

```
sfps  = Slider(axfps,  "FPS", 10, 60, valinit=fps["value"], valstep=1)
sspd  = Slider(axspd,  "speed", 0.2, 3.0, valinit=sim_speed["value"], valstep=0.05)
sdti  = Slider(axdti,  "dt_int", 0.002, 0.05, valinit=dt_int["value"], valstep=0.001)
ssett = Slider(axsett, "bif settle", 0.1, 1.5, valinit=bif.settle_s, valstep=0.05)
stran = Slider(axtran, "bif transient", 0.0, 8.0, valinit=bif.transient_s, valstep=0.1)
ssam  = Slider(axsam,  "bif sample", 0.5, 12.0, valinit=bif.sample_s, valstep=0.1)

# Radios + checks (right column). No overlap with sliders now.
ax_radio_x = fig.add_axes([0.84, 0.05, 0.14, 0.16])
ax_radio_y = fig.add_axes([0.84, 0.22, 0.14, 0.07])
ax_checks  = fig.add_axes([0.84, 0.30, 0.14, 0.06])

radio_x = RadioButtons(ax_radio_x, BifurcationRecorder.X_KEYS, active=0)
radio_y = RadioButtons(ax_radio_y, BifurcationRecorder.Y_KEYS, active=0)
checks  = CheckButtons(ax_checks, ("Auto bif", "Show phases", "Queue X while busy"),
                    (bif.enabled, show_phases["value"], True))

queue_x_while_busy = {"value": True}

for lab in list(radio_x.labels) + list(radio_y.labels):
    lab.set_fontsize(8)

controls_visible = {"value": True}
control_axes = [
    axN, axM, axz, axf, axC, axG, axa, axs, axd, axwin,
    axfps, axspd, axdti, axsett, axtran, axsam,
    ax_radio_x, ax_radio_y, ax_checks
]

def set_controls_visible(vis: bool):
    controls_visible["value"] = bool(vis)
    for ax in control_axes:
        ax.set_visible(vis)
    fig.canvas.draw_idle()

b_controls.on_clicked(lambda _evt: set_controls_visible(not controls_visible["value"]))

# -------------- callbacks & parameter application --------------
def snapshot_now(A: float, R: float):
    return {
        "C": float(params.C),
        "G": float(params.G),
        "ζ": float(params.zeta),
        "N": float(sim.N),
        "M": float(params.M),
        "f0": float(params.f0_hz),
        "A": float(A),
        "R": float(R),
    }

def current_x_value():
    k = bif.x_key
    if k == "C": return float(params.C)
    if k == "G": return float(params.G)
    if k == "ζ": return float(params.zeta)
    if k == "N": return float(sim.N)
    if k == "M": return float(params.M)
    if k == "f0": return float(params.f0_hz)
    return np.nan

def current_y_value(A_val: float, R_val: float):
    return float(R_val) if bif.y_key == "R" else float(A_val)

def mark_bif_pending():
    bif.mark_changed(sim.t, current_x_value())

def apply_params_no_reset():
    params.M = float(sM.val)
    params.zeta = float(sz.val)
    params.f0_hz = float(sf.val)
    params.C = float(sC.val)
    params.G = float(sG.val)
    params.alpha = float(sa.val)

def full_reset():
    nonlocal head
    params.M = float(sM.val)
    params.zeta = float(sz.val)
    params.f0_hz = float(sf.val)
    params.C = float(sC.val)
    params.G = float(sG.val)
    params.alpha = float(sa.val)
    params.sigma = float(ssig.val)
    params.detune = float(sdet.val)

    sim.reset(int(sN.val))
```

```
    refresh_display_idx()

    head = 0
    t_hist[:] = np.nan
    x_hist[:] = np.nan
    a_hist[:] = np.nan
    r_hist[:] = np.nan

# --- buttons
def on_pause(_evt):
    paused["value"] = not paused["value"]
    b_pause.label.set_text("Play" if paused["value"] else "Pause")
    fig.canvas.draw_idle()

def on_reset(_evt):
    full_reset()
    fig.canvas.draw_idle()

def on_clear(_evt):
    bif.clear()
    bif_sc.set_offsets(np.empty((0, 2)))
    bif_last_count["n"] = 0
    fig.canvas.draw_idle()

def on_record(_evt):
    A, _Psi = amp_phase(sim.X, sim.V, params.Omega)
    R, _Phi = order_param(sim.theta)
    bif.manual_add(snapshot_now(A, R))
    xs, ys = bif.projected_xy()
    bif_sc.set_offsets(np.column_stack([xs, ys]) if len(xs) else np.empty((0, 2)))
    bif_last_count["n"] = len(bif.points)
    fig.canvas.draw_idle()

b_pause.on_clicked(on_pause)
b_reset.on_clicked(on_reset)
b_clear.on_clicked(on_clear)
b_snap.on_clicked(on_record)

# --- radios
def on_radio_x(label):
    bif.set_axes(x_key=label)
    ax_bif.set_xlabel(label)
    set_bif_x_limits_for_key(label)
    xs, ys = bif.projected_xy()
    bif_sc.set_offsets(np.column_stack([xs, ys]) if len(xs) else np.empty((0, 2)))
    fig.canvas.draw_idle()

def on_radio_y(label):
    bif.set_axes(y_key=label)
    ax_bif.set_ylabel(label)
    set_bif_y_limits_for_key(label)
    xs, ys = bif.projected_xy()
    bif_sc.set_offsets(np.column_stack([xs, ys]) if len(xs) else np.empty((0, 2)))
    fig.canvas.draw_idle()

radio_x.on_clicked(on_radio_x)
radio_y.on_clicked(on_radio_y)

# --- checks
def on_checks(label):
    if label == "Auto bif":
        bif.enabled = not bif.enabled
    elif label == "Show phases":
        show_phases["value"] = not show_phases["value"]
        phase_sc.set_visible(show_phases["value"])
        op_line.set_visible(show_phases["value"])
        op_pt.set_visible(show_phases["value"])
    elif label == "Queue X while busy":
        queue_x_while_busy["value"] = not queue_x_while_busy["value"]
    fig.canvas.draw_idle()

checks.on_clicked(on_checks)

# --- perf sliders
def on_fps(_v):
    fps["value"] = float(sfps.val)
    if anim.event_source is not None:
        anim.event_source.interval = int(1000.0 / max(1.0, fps["value"]))

sfps.on_changed(on_fps)
sspd.on_changed(lambda _v: sim_speed.__setitem__("value", float(sspd.val)))
sdti.on_changed(lambda _v: dt_int.__setitem__("value", float(sdti.val)))
swin.on_changed(lambda _v: history_window_s.__setitem__("value", float(swin.val)))

# --- bif sliders
ssett.on_changed(lambda _v: setattr(bif, "settle_s", float(ssett.val)))
stran.on_changed(lambda _v: setattr(bif, "transient_s", float(stran.val)))
```

```
    ssam.on_changed(lambda _v: setattr(bif, "sample_s", float(ssam.val)))

    def queue_or_apply_xparam(key: str, val: float, needs_reset: bool):
        """
        If the changed parameter is the current bif.x_key AND Auto bif is enabled AND queue mode is on:
          - if recorder busy, queue the value (do NOT apply yet)
          - if recorder idle, apply immediately and mark capture
        Otherwise: apply immediately and mark capture.
        """
        val = float(val)

        is_current_x = (bif.x_key == key)
        recorder_busy = (bif._state != "idle")

        if bif.enabled and queue_x_while_busy["value"] and is_current_x and recorder_busy:
            x_queue["key"] = key
            x_queue["val"] = val
            x_queue["needs_reset"] = bool(needs_reset)
            return

        # apply now
        if needs_reset:
            full_reset()
        else:
            apply_params_no_reset()
        mark_bif_pending()

    # reset-required (omega resample)
    sN.on_changed(lambda v: queue_or_apply_xparam("N", v, needs_reset=True))
    ssig.on_changed(lambda v: queue_or_apply_xparam("σ", v, needs_reset=True))  # not an x_key, but still resets
    sdet.on_changed(lambda v: queue_or_apply_xparam("detune", v, needs_reset=True))  # not an x_key, but still resets

    # live-ish updates (some can be bif-x keys)
    sM.on_changed(lambda v: queue_or_apply_xparam("M", v, needs_reset=False))
    sz.on_changed(lambda v: queue_or_apply_xparam("ζ", v, needs_reset=False))
    sf.on_changed(lambda v: queue_or_apply_xparam("f0", v, needs_reset=False))
    sC.on_changed(lambda v: queue_or_apply_xparam("C", v, needs_reset=False))
    sG.on_changed(lambda v: queue_or_apply_xparam("G", v, needs_reset=False))
    sa.on_changed(lambda _v: (apply_params_no_reset(), mark_bif_pending()))  # alpha is not an x_key here

    # Also apply sigma/detune immediately (they are not bif-x selectable, but they matter)
    def on_sigma_change(_v):
        params.sigma = float(ssig.val)
        full_reset()
        mark_bif_pending()
    def on_detune_change(_v):
        params.detune = float(sdet.val)
        full_reset()
        mark_bif_pending()
    # Overwrite earlier hookups to keep correct behavior
    ssig.on_changed(on_sigma_change)
    sdet.on_changed(on_detune_change)

    # -------------- animation update --------------
    def update(_frame):
        nonlocal head

        if paused["value"]:
            return ()

        # If the recorder is idle and we have a queued change for the current bif-x key, apply it now.
        if bif.enabled and queue_x_while_busy["value"] and (bif._state == "idle") and (x_queue["val"] is not None) and (x_queue["key"] == bif.
x_key):
            # commit queued value
            if x_queue["key"] == "C":
                sC.set_val(float(x_queue["val"]))
                apply_params_no_reset()
            elif x_queue["key"] == "G":
                sG.set_val(float(x_queue["val"]))
                apply_params_no_reset()
            elif x_queue["key"] == "ζ":
                sz.set_val(float(x_queue["val"]))
                apply_params_no_reset()
            elif x_queue["key"] == "N":
                sN.set_val(float(x_queue["val"]))
                full_reset()
            elif x_queue["key"] == "M":
                sM.set_val(float(x_queue["val"]))
                apply_params_no_reset()
            elif x_queue["key"] == "f0":
                sf.set_val(float(x_queue["val"]))
                apply_params_no_reset()

            x_queue["key"] = None
            x_queue["val"] = None
            x_queue["needs_reset"] = False
            mark_bif_pending()
```

```
dt_frame = sim_speed["value"] / max(1.0, fps["value"])
sim.step(dt_frame, dt_int=dt_int["value"])

# measures
A, _Psi = amp_phase(sim.X, sim.V, params.Omega)
R, Phi = order_param(sim.theta)

# history ring buffer
t_hist[head] = sim.t
x_hist[head] = sim.X
a_hist[head] = A
r_hist[head] = R
head = (head + 1) % len(t_hist)

ordered = np.roll(np.arange(len(t_hist)), -head)
tt = t_hist[ordered]
valid = ~np.isnan(tt)
tt = tt[valid]
xx = x_hist[ordered][valid]
aa = a_hist[ordered][valid]
rr = r_hist[ordered][valid]

if len(tt) >= 2:
    tplot = tt - tt[0]

    # window the view (keeps oscillation graphs readable)
    win = float(history_window_s["value"])
    tmax = tplot[-1]
    keep = tplot >= max(0.0, tmax - win)

    tW = tplot[keep]
    xW = xx[keep]
    aW = aa[keep]
    rW = rr[keep]

    if len(tW) >= 2:
        # decimate to keep draw cost sane
        stride = max(1, len(tW) // 700)
        tD = tW[::stride]
        xD = xW[::stride]
        aD = aW[::stride]
        rD = rW[::stride]

        line_x.set_data(tD, xD)
        line_ap.set_data(tD, aD)
        line_am.set_data(tD, -aD)
        line_r.set_data(tD, rD)

        ax_xtime.set_xlim(max(0.0, tD[-1] - win), max(1e-6, tD[-1]))
        ax_rtime.set_xlim(max(0.0, tD[-1] - win), max(1e-6, tD[-1]))

        m = float(np.max(np.abs(np.concatenate([xD, aD]))))
        m = max(m, 0.05)
        ax_xtime.set_ylim(-1.15*m, 1.15*m)

# phase circle
if show_phases["value"]:
    idx = display_idx["idx"]
    th = sim.theta[idx]
    pts = np.column_stack([np.cos(th), np.sin(th)])
    phase_sc.set_offsets(pts)
    cx, cy = R * np.cos(Phi), R * np.sin(Phi)
    op_line.set_data([0.0, cx], [0.0, cy])
    op_pt.set_data([cx], [cy])

# adaptive bifurcation capture
snap = snapshot_now(A, R)
bif.tick_and_maybe_record(sim.t, snap, current_y_value(A, R))

# only redraw bif scatter if changed
if len(bif.points) != bif_last_count["n"]:
    xs, ys = bif.projected_xy()
    bif_sc.set_offsets(np.column_stack([xs, ys]) if len(xs) else np.empty((0, 2)))
    bif_last_count["n"] = len(bif.points)

    # If y is A, gently expand ylim if needed
    if bif.y_key == "A" and len(ys):
        ymax = float(np.max(ys))
        lo, hi = ax_bif.get_ylim()
        if ymax > 0.95 * hi:
            ax_bif.set_ylim(0.0, max(0.25, 1.25 * ymax))

Nc = critical_crowd_size(params)
pending_txt = ""
if bif.enabled and queue_x_while_busy["value"] and (x_queue["val"] is not None) and (x_queue["key"] == bif.x_key):
    pending_txt = f" | queued {bif.x_key}→{x_queue['val']:.3g}"
```

```
        title.set_text(
            f"t={sim.t:6.2f} | X={sim.X:+.4f} A={A:.4f} R={R:.3f} | "
            f"N={sim.N} M={params.M:.2f} f0={params.f0_hz:.2f}Hz ζ={params.zeta:.3f} "
            f"C={params.C:.2f} G={params.G:.3f} Nc≈{Nc:.1f} | bif:{bif._state}{pending_txt}"
        )

        xcrit = critical_value_for_xkey(bif.x_key)
        if np.isfinite(xcrit):
            theory_line.set_xdata([xcrit, xcrit])
            theory_line.set_visible(True)
        else:
            theory_line.set_visible(False)

        return ()
    # Kick off the bifurcation recorder immediately so it actually records something.
    mark_bif_pending()

    anim = FuncAnimation(fig, update, interval=int(1000.0 / fps["value"]), blit=False)
    plt.show()


if __name__ == "__main__":
    run_interactive()
```